Accelerating Multimodal AI for MPN Diagnosis

Final Presentation – Pair Lab, NYCU

OpenACC

More Science, Less Programming

Pair Lab

NYCU Department of Computer Science and Taipei Veterans General Hospital



Yu-Chee Tseng
Department of Computer Science, NYCU



Cheng-Kuan Lin
Department of Computer Science, NYCU



Chia-Jen Liu
Taipei Veterans General Hospital



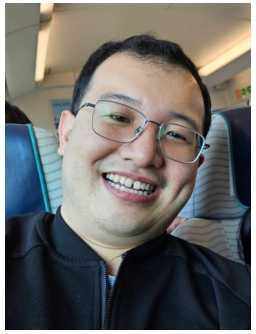
Yi-Yang Feng
Department of Computer Science, NYCU



Sung-Hao Tsai
Department of Computer Science, NYCU



Tong-Jin Chou
Department of Computer Science, NYCU



Mentor

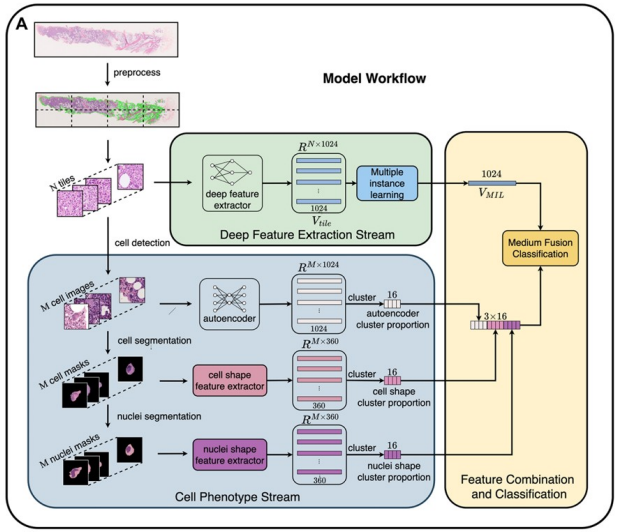
Andy Liao
Solutions Architect
NVAITC Taiwan

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Application: MPN Diagnosis

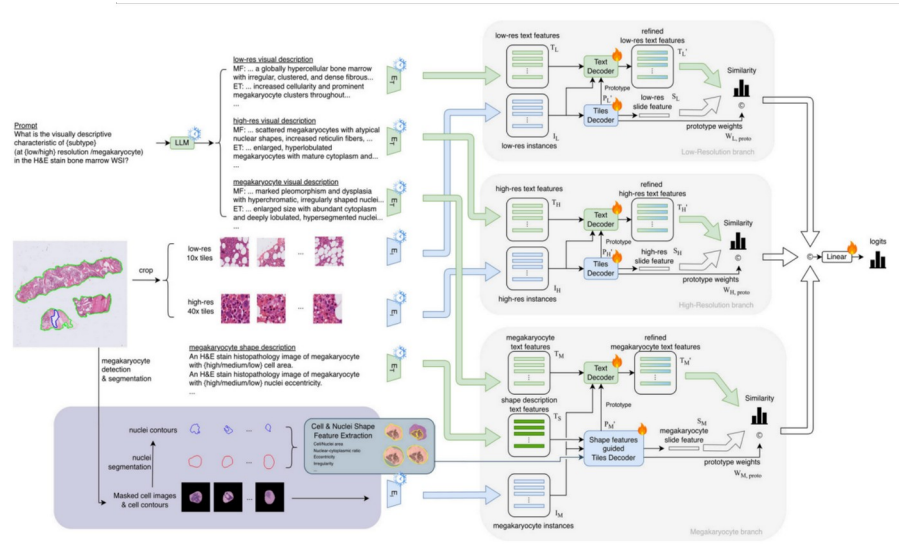
AI that diagnoses blood cancers from gigapixel bone-marrow slides: two pipelines, one task

DS-DFCPF



Bottleneck: CPU and data movement

VLM-MIL



Bottleneck: training loop waiting on the GPU

Problem: pipelines too slow to iterate on. Driver: larger experiments and quicker clinical turnaround. Motif: multiple-instance learning over tens of thousands of patches, segmentation, clustering, fusion.

Evolution and Strategy

From a plan to port stages to the GPU, to a plan to stop the GPU from waiting

Goal

- Speed up both MPN pipelines
- Keep accuracy unchanged

Initial strategy

- Profile where the time goes
- Move CPU-bound stages to the GPU
- Use AI coding assistants to profile and port code

How it changed

- Profiling showed the GPU was waiting, not slow
- Biggest wins: cut transfers, batch small ops, parallelize folds

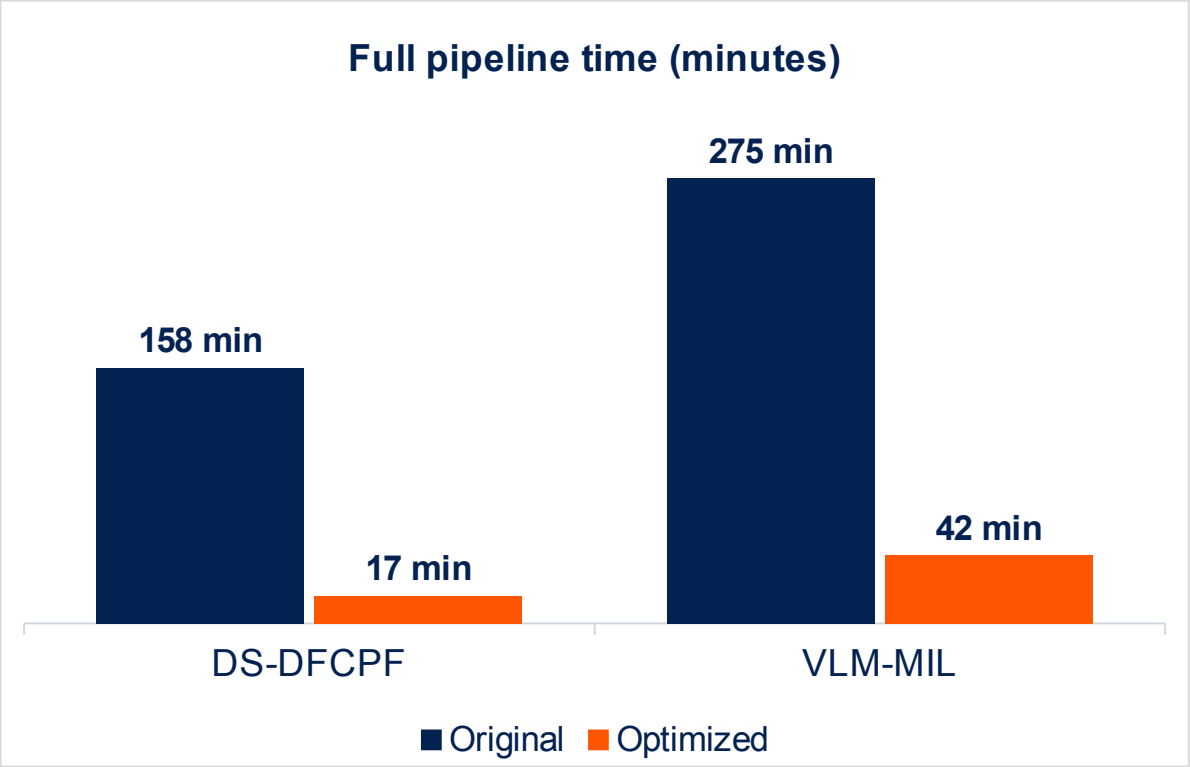
Shared Playbook

Three techniques, applied to both pipelines

Technique	DS-DFCPF	VLM-MIL
1. Cut CPU↔GPU traffic and waiting	Cache all features on the GPU once	Keep results on the GPU (syncs −88%); pinned memory (transfer 9.10 → 2.21 s)
2. Batch many small ops into one	Train 60 small models as one GPU batch (~1 h → ~3 min)	Shape loss for 15 attributes in one pass
3. Parallelize independent work	5 folds on 5 GPUs (CLAM ~1 h → ~2 min); parallel slide reading	5 folds on 5 H200s (3 h 27 min → 42 min)

Results and Final Profile (1/2)

Same accuracy, much shorter turnaround



9.3x

DS-DFCPF: ~2.6 h → ~17 min

6.5x

VLM-MIL: 4 h 35 min → 42 min

Accuracy unchanged: DS-DFCPF ~0.69 → ~0.69; VLM-MIL identical to 4 decimals. Baseline: original code on one H200.

Results and Final Profile (2/2)

The GPU now does more work; multi-core CPU code moved to the GPU



Step	CPU	GPU	Speedup
Cell shape features	4.61 s	3.05 s	**1.5x**
Cell nucleus features	20.85 s	5.86 s	**3.6x**
Fusion model	56 min	5.5 min	**10x**

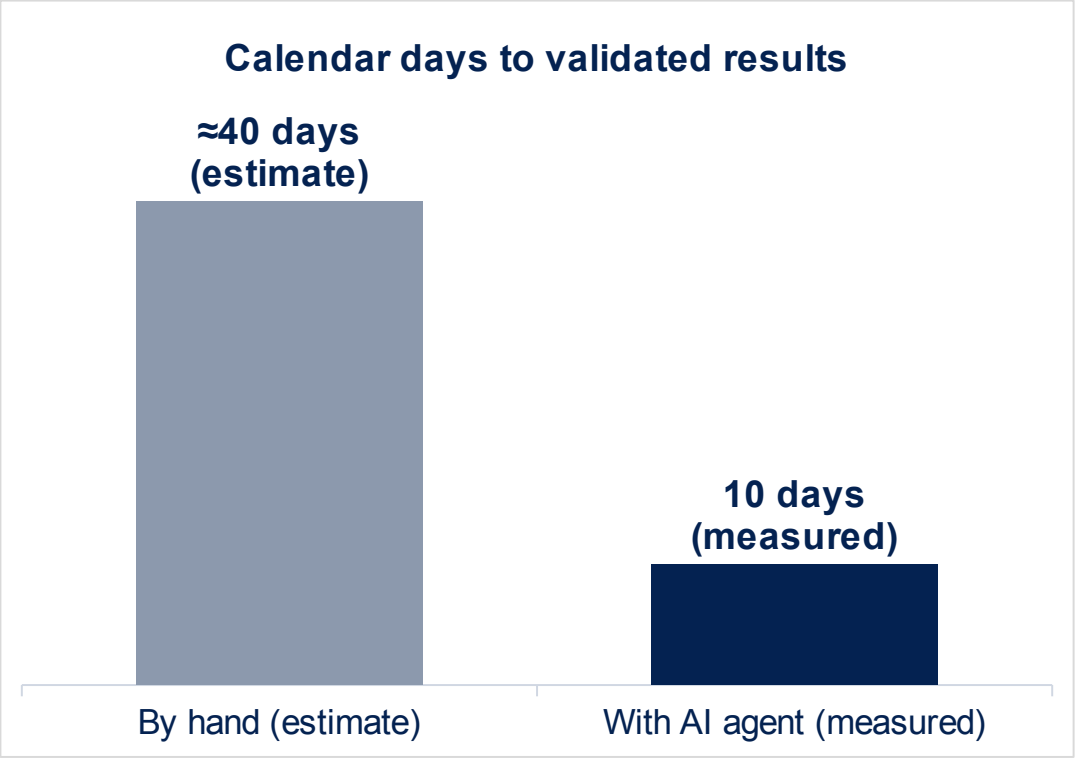
Before: CLAM training, GPU busy 8.7%, 17.1 ms per iteration.

After caching features on the GPU: GPU busy 36.5%, 4.1 ms per iteration.

- **Nvidia tool : CuPy , CuCIM**
- **Learned:** the GPU was idle, not slow; waiting was the bottleneck

AI Agent/nsight agent: Workflow Speed-up

(DS-DFCPF only) Time to validated results, not program run time



Measured

- **143** Slurm jobs, 39.8 GPU-hours
- **251** prompts over 10 days (Oct 18–27)
- **9** optimized variants, each checked against the original
- **21** validation and comparison scripts
- **≈4x** shorter calendar time than the by-hand estimate (range 2–7x)

By-hand estimate: about 226 person-hours at 40 h/week (range 122–393), counting the same work we actually did, including bit-exact CuPy and the patch-size study. No control group.

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

AI Agent: Always-on Monitoring and Lessons

(DS-DFCPF only) Why an agent helps beyond saving hours

Always on

- A person cannot watch jobs 24/7. The agent can.
- **143** jobs, 17 resubmissions (9 right after a failure)
- Checked job states with sacct through a ~30-min scheduler outage, then resubmitted

What worked and what didn't

- **Worked:** validation first; context kept in files (README, change log, backups, git)
- **People decide:** we asked for version control and a fair re-measurement
- **Didn't:** the agent repeated a documented path error and guessed one bottleneck wrong

Our takeaway: working with an AI agent needs strict version control and a README, and people to help keep both maintained.

Energy Efficiency

Estimated with the official calculator (H200 nodes)

DS-DFCPF — Node power efficiency 1.2x

INPUTS		
Baseline	GPU	*Select the baseline as GPU or CPU*
Baseline GPU Type	8x H200 141GB SXM5	*Select the CPU/GPU Type or GPU Node configuration*
Baseline GPU # GPUs	1	*Select the number of CPU cores or GPUs (within one node)*
Final GPU Node	8x H200 141GB SXM5	*Select the final GPU Type or GPU Node configuration*
Final # GPUs	8	*Select the number of GPUs (within one node)*
Application Speedup	9.3x	*Enter the amount of achieved speedup*
Node Replacement	1.2x	

GPU NODE POWER SAVINGS	Baseline 8x H200 141GB SXM5	Comparison 8x H200 141GB SXM5	Power Savings
Compute Power (W)	11,971	10,298	1,673
Networking Power (W)	1,023	880	143
Total Power (W)	**12,994**	**11,178**	1,816
Node Power efficiency	**1.2x**		

Baseline 1 GPU → 8 GPUs, speedup 9.3x. About 15,900 kWh per year saved per node.

VLM-MIL — Node power efficiency 1.3x

INPUTS		
Baseline	GPU	*Select the baseline as GPU or CPU*
Baseline GPU Type	8x H200 141GB SXM5	*Select the CPU/GPU Type or GPU Node configuration*
Baseline GPU # GPUs	1	*Select the number of CPU cores or GPUs (within one node)*
Final GPU Node	8x H200 141GB SXM5	*Select the final GPU Type or GPU Node configuration*
Final # GPUs	5	*Select the number of GPUs (within one node)*
Application Speedup	6.5x	*Enter the amount of achieved speedup*
Node Replacement	1.3x	

GPU NODE POWER SAVINGS	Baseline 8x H200 141GB SXM5	Comparison 8x H200 141GB SXM5	Power Savings
Compute Power (W)	13,387	10,298	3,089
Networking Power (W)	1,144	880	264
Total Power (W)	**14,531**	**11,178**	3,353
Node Power efficiency	**1.3x**		

Baseline 1 GPU → 5 GPUs, speedup 6.5x. About 29,400 kWh per year saved per node.

Most of our speedup comes from using more GPUs at once, so energy savings (1.2–1.3x) are smaller than time savings. Estimates only.

Open the calculator

What Problems Have We Encountered?

What learning GPUs and NVIDIA tools taught us

Category	Challenge	What we learned and did
Legacy app structure	CPU waited on the GPU: 78,837 sync calls	Found with Nsight; kept results on GPU (−88% syncs)
Algorithms	CPU/GPU floats differ; large batches did not converge	Checked epoch 1 (diff 6e-8); TF32 only
Tool surprises	non_blocking needs pinned memory; CuPy needs NVRTC path; kernels fuse multiply-adds	Pinned memory first; set library path; explicit-rounding intrinsics
System setup	12 CPUs per GPU; folds must map to the GPUs a job gets	Assigned folds via CUDA_VISIBLE_DEVICES

Wishlist

What we still needed after using NVIDIA's tools

1

Tools

Live monitoring and alerts: one slow node (180 s vs 28 s) went unnoticed until the job ended

2

Language standards / libraries

SOM clustering in cuML, so the pipeline stays on the GPU

3

Event

Follow-up sessions with NVIDIA engineers on open issues, such as the slow node

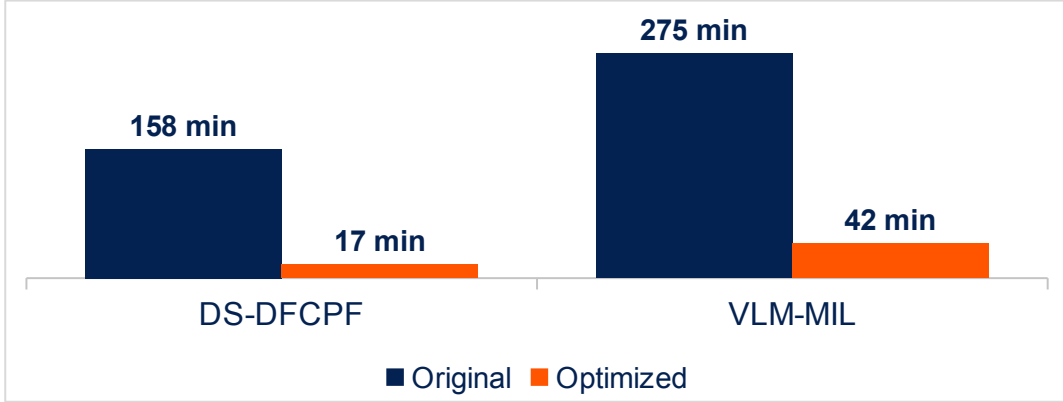
4

Systems

GPU utilization and job progress on the cluster dashboard

Application Background

AI models that diagnose myeloproliferative neoplasms (blood cancers) from gigapixel bone-marrow slides.
Two models: DS-DFCPF (deep features + cell phenotypes) and VLM-MIL (vision-language multiple-instance learning).
Motifs: multiple-instance learning over tens of thousands of patches, cell segmentation, clustering, small-model fusion.



Full pipeline time, original vs. optimized (minutes).

Hackathon Objectives and Approach

Programming models: PyTorch, CuPy, multi-GPU Slurm jobs
Profiling/hot spots: Nsight Systems + NVTX; GPU busy only ~1–13%
Refactoring: merged stages, features kept on the GPU, batched small ops
Libraries: CuPy; CuCIM ;TF32; pinned memory
Performance tuning: parallel folds, multi-process I/O
Other: AI coding assistants for profiling and porting

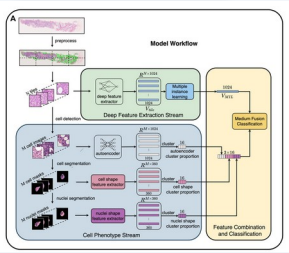
Technical Accomplishments and Impact

DS-DFCPF ~2.6 h → ~17 min (9.3x); VLM-MIL 4 h 35 min → 42 min (6.5x).
How: cut CPU↔GPU traffic, batch small operations, parallelize independent folds and I/O.
Accuracy unchanged (~0.69; VLM-MIL identical to 4 decimals).
Why it matters: faster experiments and quicker turnaround toward clinical use.

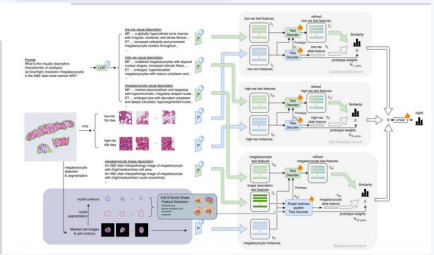
讓 AI 更快診斷血癌：骨髓切片分析加速

Pair Lab 團隊來自陽明交大曾煜棋老師帶領的 Pair 實驗室，
AI Agent 將 DS-DFCPF 工作流程加速了約 4 倍（估計）；GPU 將 MPN 診斷研究加速了 6.5–9.3 倍！！

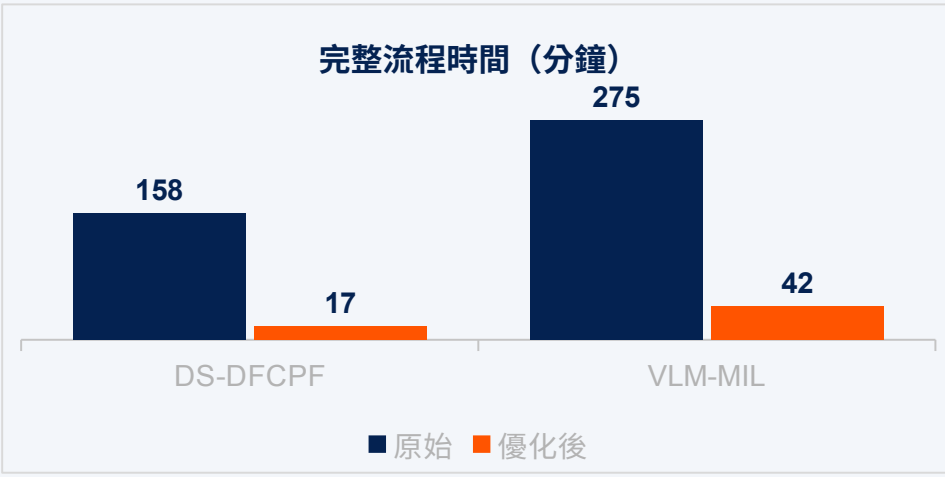
DS-DFCPF



VLM-MIL



骨髓增生性腫瘤（MPN）是一類血癌，診斷需要病理醫師在超高解析度的骨髓切片上逐一判讀細胞，耗時且仰賴經驗。我們開發兩條 AI 診斷流程（DS-DFCPF、VLM-MIL）協助自動分類，但原本一次完整實驗要數小時，難以反覆調整與擴大規模。這次黑客松，我們用 Nsight Systems 找出瓶頸：GPU 常在等 CPU 與資料搬移。因此我們把 CPU 運算移到 GPU、減少 CPU 與 GPU 之間的往返，並把許多小工作合併或平行執行。結果 DS-DFCPF 由 158 分鐘縮短到 17 分鐘，VLM-MIL 由 4 小時 35 分縮短到 42 分鐘，準確度不變。實驗更快，研究者就能更常驗證想法、使用更大的資料，也為縮短臨床回報時間打下基礎。



9.3x
DS-DFCPF
GPU 加速

6.5x
VLM-MIL
GPU 加速

≈4x
AI Agent 工作流程
（估計）

準確度不變：DS-DFCPF ≈0.69 → ≈0.69；VLM-MIL 小數點後四位相同。AI Agent：實測 10 天，人工估計約 40 天（約 226 人時，無對照組）。

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA


Thank You
OpenACC
More Science, Less Programming

Final Thoughts

What we take away and what comes next

1

Was it worth it?

Two pipelines now run 6.5–9.3x faster end to end, with accuracy unchanged.

2

Will we continue?

Yes: we plan to keep looking for speedups in both models.

3

Next steps

Re-profile both pipelines and tackle the next bottleneck, starting with DS-DFCPF's autoencoder steps.

Backup: DS-DFCPF Stage by Stage

About 2.6 h → ~17 min

Stage	What we did	Playbook	Time
CLAM training	Cache features on the GPU; 5 folds on 5 GPUs at once	Cut traffic Parallelize	**~1 h → ~2 min**
Feature export	Read data once, shared by all 5 models	Cut traffic	**~2 min → ~20 s**
Cell feature extraction	Merge two stages; features and preprocessing on the GPU; more parallel chunks	Pipeline-specific	**~20 min → ~7 min**
Autoencoder: rendering	Multi-process parallel slide reading	Parallelize	**~1 h → ~5 min**
Autoencoder: training	TF32, fewer checkpoints, parallel file writing	Pipeline-specific	**~35 min → ~8 min**
Fusion model	Batch many small models into one GPU run	Batch	**~1 h → ~3 min**

Whole pipeline in one job (no re-queuing between steps): 158 min → 17 min.

Backup: VLM-MIL Changes

4 h 35 min → 42 min on H200

Change	Before	Now	Effect
1. Keep results on the GPU Cut traffic	Every iteration copies loss and predictions to the CPU and waits	Accumulate on the GPU; bring totals back once per epoch	**Sync calls 78,837 → 9,601 (−88%)**
2. Shape loss in one pass Batch	For each of 15 attributes: pick labeled cells, ask for a count, wait	Whole cells × 15 table at once; unlabeled (−1) skipped	**Loss identical to the original**
3. Non-blocking transfer Cut traffic	CPU waits for each copy to progress	CPU moves on; the copy runs in the background	**No CPU stalls on copies (needs 4)**
4. Pinned memory Cut traffic	Data is first copied to a staging buffer	Data loads into pinned memory; hardware takes it directly	**Transfer time 9.10 → 2.21 s**
5. 5-fold parallel Parallelize	Folds trained one after another	Each fold on its own H200 at the same time	**3 h 27 min → 42 min (4.9x)**

Baseline: 8 h 20 min on a local RTX 3090 → 4 h 35 min on H200 → 3 h 27 min after the first four changes.

Backup: What We Tried but Did Not Adopt

DS-DFCPF: every change was validated, not just fast

Parallel tuning of the fusion model

No speedup: the models are small and time goes to fixed overhead

Large-batch autoencoder training

Did not converge; accuracy dropped

bf16 mixed precision

Only a small gain, less than TF32

We kept only the changes that preserve accuracy.